CSCE 5300: Introduction to Big Data and Data Science

Chapter 6 (6.1): State-of-the-Art Big Data Analytics Architectures and Tools:analytics Genetic Algorithms

Qutline

- **6.1.1. Introduction**
- **6.1.2. Foundations of Genetic Algorithms**
- **6.1.3. Search space and fitness score**
- **6.1.4. Operators of Genetic Algorithms**
- **6.1.5. Complexity of Genetic Algorithms**
- **6.1.6. Examples**
- **6.1.7. Advantages and limitations of Genetic Algorithms**
- **6.1.8. Applications of Genetic Algorithms**

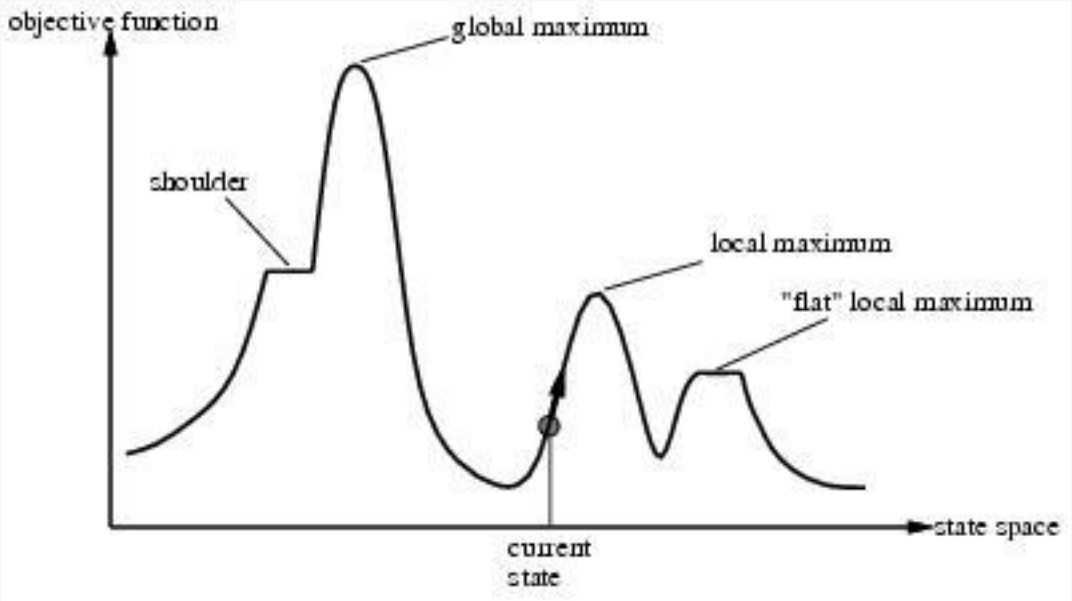
6.1.1. Introduction

- **Local search and optimization**

- Local search= use single current state and moves to neighboring states (Fig. 6.1.1).
- Advantages:
  - Uses very little memory
  - Often finds reasonable solutions in large or infinite state spaces.
- Are also useful for pure optimization problems.
  - Find best state according to some *objective function*.
  - e. g., survival of the fittest as a metaphor for optimization.

6.1.1. Introduction

- Fig. 6.1.1.



6.1.1. Introduction

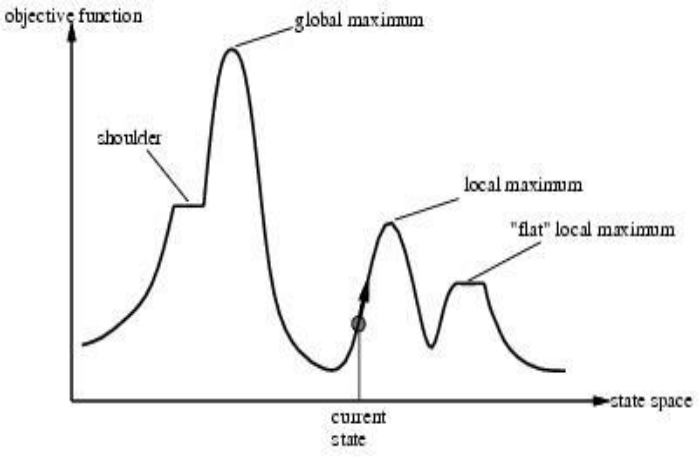
- **Advantages of local search**

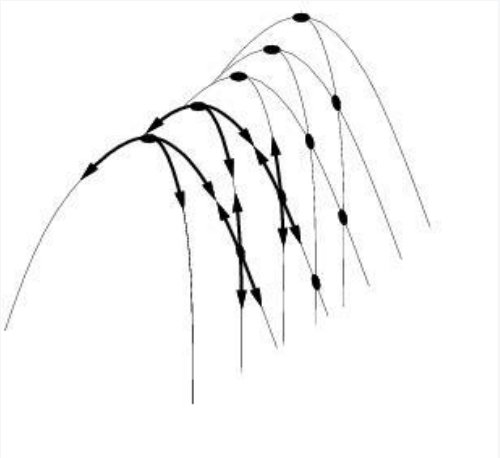
- The runtime of min-conflicts is roughly independent of problem size.
  - Solving the millions-queen problem in roughly 50 steps.

- Local search can be used in an online setting.
  - Backtrack search requires more time

6.1.1. Introduction

- **Drawbacks of Local Search (Fig. 6.1.2)**





- Fig. 6.1.2.

6.1.1. Introduction

- Ridge = sequence of local maxima difficult for greedy algorithms to navigate
- Plateau = an area of the state space where the evaluation function is flat.
- Gets stuck 86% of the time.

- **However**
- It takes only 4 steps on the average when it succeeds
- Takes an average of 3 steps when it gets stuck in local minimum
- Not bad for a problem with $8^8 \approx 17$ million states

6.1.1. Introduction

- **Genetic Algorithms (GAs)** are adaptive heuristic (strategies derived from previous experience with similar problems) search algorithms that belong to the larger part of evolutionary algorithms.

- **GAs** is a search-based optimization technique using principles of **Genetics and Natural Selection**.
- These are intelligent exploitation of random search provided with historical data to direct the search into the region of better performance in solution space [1].

- ***GAs were developed by John Holland and his students and colleagues at the University of Michigan***, most notably ***David E. Goldberg and has since been tried on various optimization problems*** with a high degree of success.

- [1] John H. Holland, *Adaptation in Natural and Artificial Systems. An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence,* The MIT Press, Cambridge, Massachusetts, 1992.

6.1.1. Introduction

- **Some basic terminology of GAs**

- **Population** − It is a subset of all the possible (encoded) solutions to the given problem.
- The population for a GA is analogous to the population for human beings except that instead of human beings, we have Candidate Solutions representing human beings.
- **Chromosomes** − A chromosome is one such solution to the given problem.
- **Gene** − A gene is one element position of a chromosome.
- **Genotype** − Genotype is the population in the computation space.
- In the computation space, the solutions are represented in a way which can be easily understood and manipulated using a computing system.
- **Allele** − It is the value a gene takes for a particular chromosome.

6.1.1. Introduction

- **Fitness Function** − The fitness function is a function which takes a **candidate solution to the problem as input and produces as output** how “fit” our how “good” the solution is with respect to the problem in consideration.
- ***Calculation of fitness value is done repeatedly in a GA and therefore it should be sufficiently fast.***
- ***A slow computation*** of the fitness value can adversely affect a GA and make it exceptionally slow.
- ***In most cases the fitness function and the objective function are the sa***me as the objective is to either maximize or minimize the given objective function.
- ***However, for more complex problems with multiple objectives and constraints, one might choose to have a different fitness function.***

6.1.1. Introduction

- ***A fitness function should possess the following characteristics –***

- The fitness function should be sufficiently fast to compute.
- It must quantitatively measure how fit a given solution is or how fit individuals can be produced from the given solution.
- In some cases, ***calculating the fitness function directly might not be possible*** due to the inherent complexities of the problem at hand.
- In such cases, we can do ***fitness approximation*** to suit our needs.

6.1.1. Introduction

- **Genetic algorithms simulate the process of natural selection** which means those species who can adapt to changes in their environment are able to survive and reproduce and go to next generation.
- In simple words, they simulate “survival of the fittest” among individual of consecutive generation for solving a problem.

- **Each generation (iteration) consists of a population of individuals** and each individual represents a point in search space and possible solution.

- Each individual is represented as a string of characters/integers/floats/bits.
- This string is analogous to the Chromosome.

6.1.1. Introduction

- **Why use GAs**
- They are Robust.
- Provide optimization over large space state.
- In contrast to other analogs, they do not break on slight change in input or presence of noise.

- GAs have the ability to deliver a “good-enough” solution “fast-enough”.
- This makes GAs attractive for use in solving optimization problems.

6.1.1. Introduction

- **Where GAs can be used –**

- **Solving Difficult Problems**
- In computer science, there is a large set of problems, which are *NP-Hard (an NP-hard problem is a computational problem that is at least as hard as any problem in the NP class (non-deterministic polynomial time)).*

- What this essentially means is that, even the most powerful computing systems take a very long time (even years!) to solve that problem.

- In such a scenario, GAs prove to be an efficient tool to provide **usable near-optimal solutions** in a short amount of time.

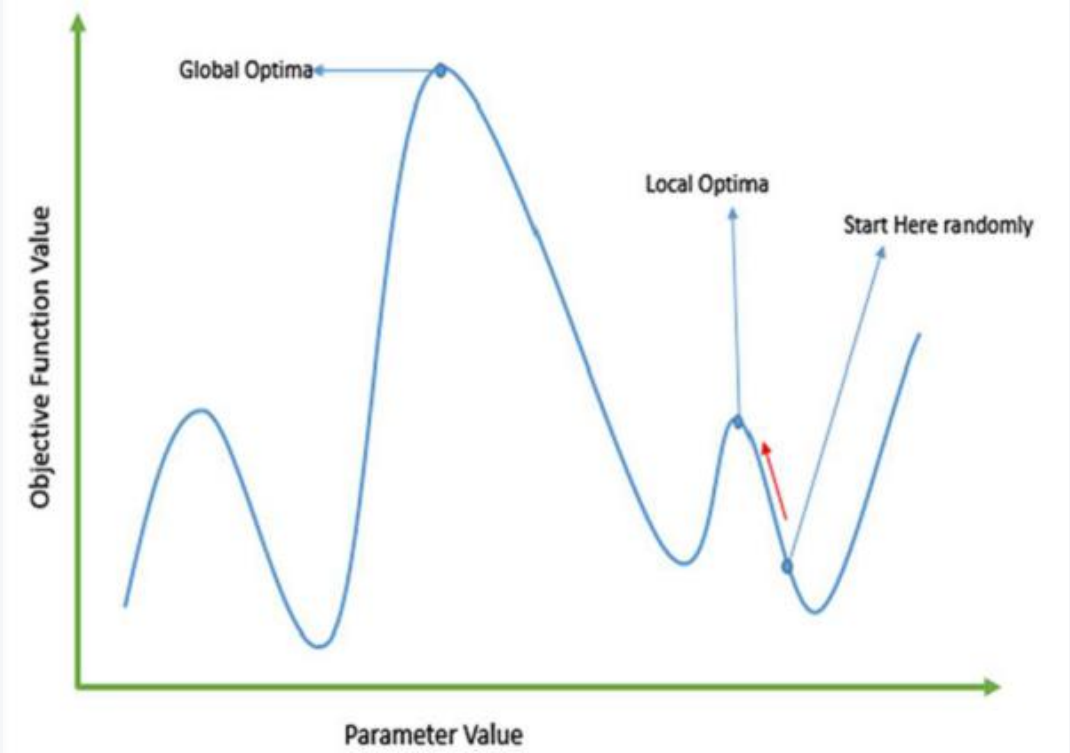
6.1.1. Introduction

- **Failure of Gradient Based Methods**
- Traditional calculus-based methods work by starting at a random point and by moving in the direction of the gradient, till we reach the top of the hill.

- This technique is efficient and works very well for single-peaked objective functions like the cost function in linear regression.

- But, in most real-world situations, we have a very complex problem called as landscapes, which are made of many peaks and many valleys, which causes such methods to fail, as they suffer from an inherent tendency of getting stuck at the local optima as shown in the Fig. 6.1.3.

6.1.1. Introduction



- Fig. 6.1.3.

6.1.1. Introduction

- **Getting a Good Solution Fast**
- Some difficult problems like the ***Travelling Salesperson Problem (TSP), have real-world applications like path finding and VLSI Design.***

- Now imagine that you are using your ***GPS Navigation system, and it takes a few minutes (or even a few hours)*** to compute the “optimal” path from the source to destination.

- Delay in such real-world applications is not acceptable and therefore a “good-enough” solution, which is delivered “fast” is what is required.

6.1.2. Foundations of Genetic Algorithms

- In GAs, we have a **pool or a population of possible solutions** to the given problem.
- These solutions then undergo recombination and mutation (like in natural genetics), producing new children, and the process is repeated over various generations.
- Each individual (or candidate solution) is assigned a fitness value (based on its objective function value) and the fitter individuals are given a higher chance to mate and yield more "fitter" individuals.
- This is in line with the Darwinian Theory of "Survival of the Fittest".
- In this way we keep "evolving" better individuals or solutions over generations, till we reach a stopping criterion.
- GAs are sufficiently randomized in nature, but ***they perform much better than random local search*** (in which we just try various random solutions, keeping track of the best so far), as ***they exploit historical information as well***.

6.1.2. Foundations of Genetic Algorithms

- GAs are based on an analogy with genetic structure and behavior of chromosomes of the population.
- Following is the foundation of GAs based on this analogy –

- *Individuals in population compete for resources and mate.*
- *Those individuals who are successful (fittest) then mate to create more offspring than others.*
- *Genes from "fittest" parent propagate throughout the generation, that is sometimes parents create offspring which is better than either parent.*

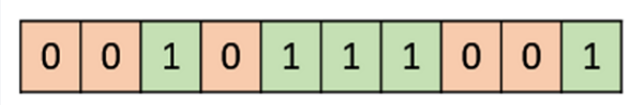
- Thus, each successive generation is more suited for their environment.

6.1.2. Foundations of Genetic Algorithms

- **Genotype Representation**

- **Binary Representation**
- This is one of the simplest and most widely used representation in GAs.
- In this type of representation, the genotype consists of bit strings (Fig. 6.1.4).

- Fig. 6.1.6.1.



6.1.2. Foundations of Genetic Algorithms

- **Real Valued Representation**

- For problems where we want to define the genes using continuous rather than discrete variables, the real valued representation is the most natural.
- The precision of these real valued or floating-point numbers is however limited in the computer (Fig. 6.1.5).

- Fig. 6.1.5.

0.5	0.2	0.6	0.8	0.7	0.4	0.3	0.2	0.1	0.9

6.1.2. Foundations of Genetic Algorithms

- **Integer Representation**
- For discrete valued genes, we cannot always limit the solution space to binary ‘yes’ or ‘no’.
- For example, if we want to encode the four distances – North, South, East and West, we can encode them as **{0,1,2,3}**.
- In such cases, integer representation is desirable (Fig. 6.1.6).

- Fig. 6.1.6.

1	2	3	4	3	2	4	1	2	1

6.1.3. Search Space and Fitness Score

- The population of individuals are maintained within search space.
- ***Each individual represents a solution in search space for given problem.***
- Each individual is coded as a finite length vector (analogous to chromosome) of components.

- These variable components are analogous to genes (variable components).
- A chromosome (individual) is composed of several genes.

6.1.3. Search Space and Fitness Score

- A Fitness Score/Value is given to each individual which **shows the ability of an individual to "compete"**.
- The individual having optimal fitness score (or near optimal) are sought.
- The GAs maintains the population of n individuals (chromosome/solutions) along with their fitness scores.
- The individuals having better fitness scores are given more chance to reproduce than others.
- The individuals with better fitness scores are selected who mate and produce **better offspring** by combining chromosomes of parents.

6.1.3. Search Space and Fitness Score

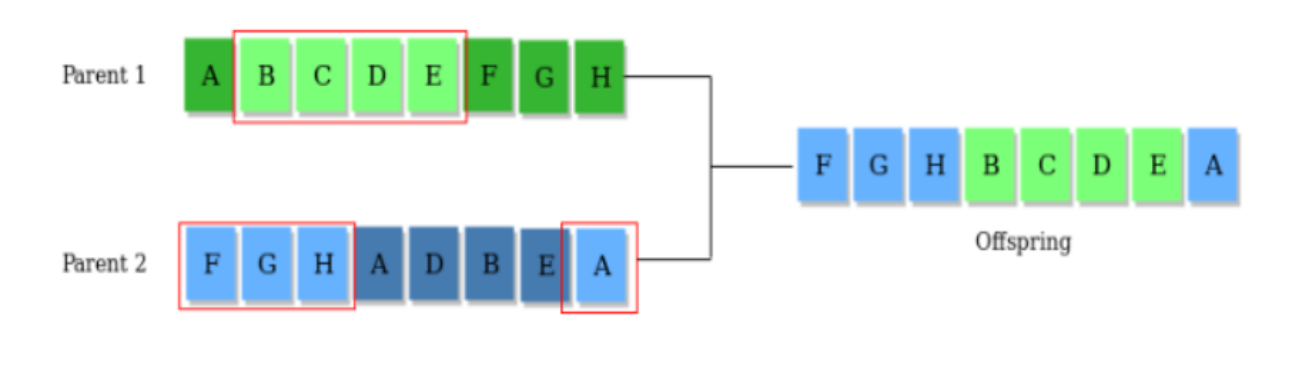
- *So, some individuals die and get replaced by new arrivals eventually creating new generation when all the mating opportunity of the old population is exhausted.*
- *It is hoped that over successive generations better solutions will arrive while least fit die.*
- *Each new generation has on average more "better genes" than the individual (solution) of previous generations.*
- Thus, each new generations have better **"partial solutions"** than previous generations.
- *Once the offspring produced having no significant difference from offspring produced by previous populations, the population is converged.*
- *The algorithm is said to be converged to a set of solutions for the problem.*

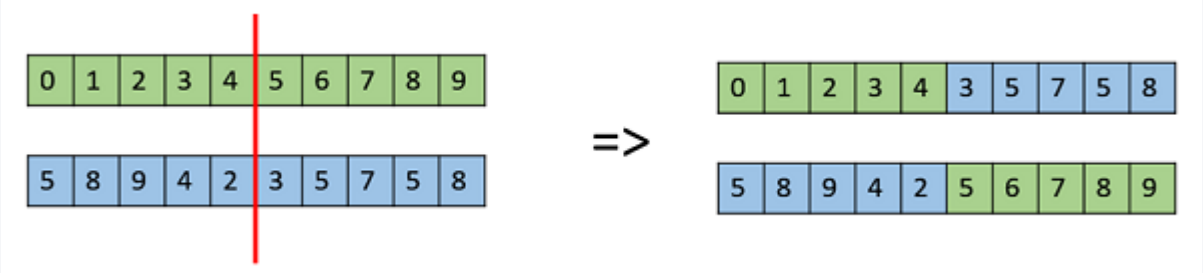
6.1.4. Operators of Genetic Algorithms

- Once the initial generation is created, the algorithm evolves the generation using following operators –
  1) Selection Operator: *The idea is to give preference to the individuals with good fitness scores and allow them to pass their genes to successive generations.*
- ***2) Crossover Operator:*** *This represents mating between individuals.*
- *Two individuals are selected using selection operator and crossover sites are chosen randomly.*
- *Then the genes at these crossover sites are exchanged thus creating a completely new individual (offspring).*
- For example (Fig. 6.1.7) –

6.1.4. Operators of Genetic Algorithms

- Fig. 6.1.7.



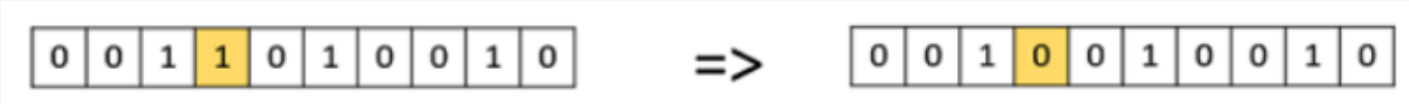


6.1.4. Operators of Genetic Algorithms

- ***3) Mutation Operator:*** *The key idea is to insert random genes in offspring to maintain the diversity in the population to avoid premature convergence.*
- *In simple terms, mutation may be defined as a small random tweak in the chromosome, to get a new solution.*
- *It is used to maintain and introduce diversity in the genetic population and is usually applied with a low probability –* p_m.
- **If the probability is very high, the GA gets reduced to a random search.**
- *Mutation is the part of the GA which is related to the "exploration" of the search space.*
- **It has been observed that mutation is essential to the convergence of the GA while crossover is not.**
- For example(Fig. 6.1.8) –

6.1.4. Operators of Genetic Algorithms

- Fig. 6.1.6.1.



6.1.4. Operators of Genetic Algorithms

- **Population Initialization**
- *There are two primary methods to initialize a population in a GA. They are −*
- ***Random Initialization*** *− Populate the initial population with completely random solutions.*
- ***Heuristic initialization*** *− Populate the initial population using a known heuristic for the problem.*

- *It has been observed that the entire population should not be initialized using a heuristic, as it can result in the population having similar solutions and very little diversity.*

6.1.4. Operators of Genetic Algorithms

- ***It has been experimentally observed that the random solutions are the ones to drive the population to optimality.***
- *Therefore, with heuristic initialization, we just seed the population with a couple of good solutions, filling up the rest with random solutions rather than filling the entire population with heuristic based solutions.*

- *It has also been observed that heuristic initialization in some cases, only effects the initial fitness of the population, but in the end, it is the diversity of the solutions which lead to optimality.*

6.1.4. Operators of Genetic Algorithms

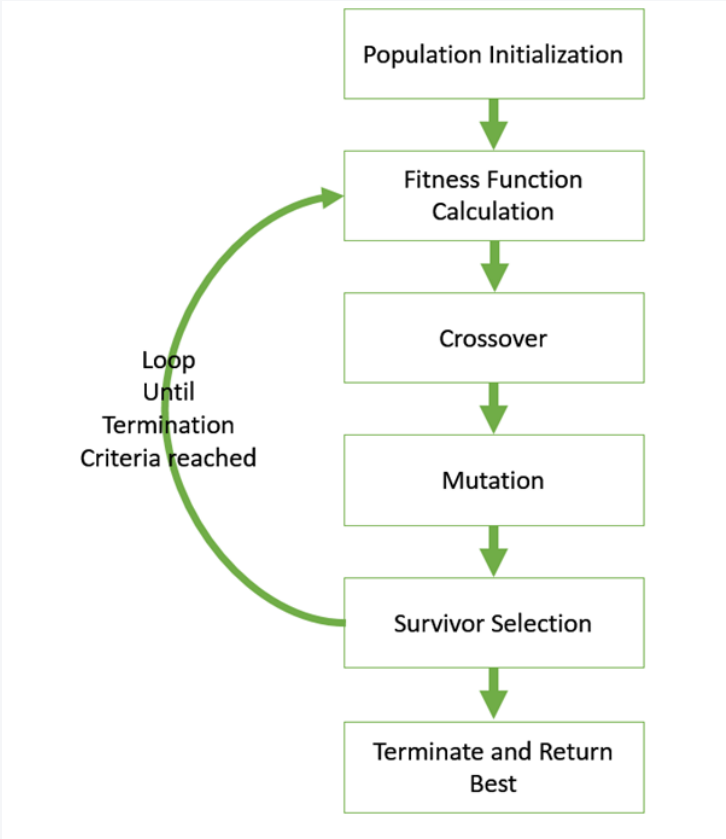
- **The whole algorithm can be summarized as –**
- 1) Randomly initialize populations p
- 2) Determine fitness of population
- 3) Until convergence repeat:
- a) Select parents from population
- b) Crossover and generate new population
- c) Perform mutation on new population
- d) Calculate fitness for new population

6.1.4. Operators of Genetic Algorithms

- **Basic Structure of GA**
- *The basic structure of a GA is as follows −*
- *We start with an initial population (which may be generated at random or seeded by other heuristics), select parents from this population for mating.*
- *Apply crossover and mutation operators on the parents to generate new off-springs. And finally, these off-springs replace the existing individuals in the population and the process repeats.*
- In this way GAs actually try to mimic the human evolution to some extent (Fig. 6.1.9).

6.1.4. Operators of Genetic Algorithms

- Fig. 6.1.9.



6.1.4. Operators of Genetic Algorithms

- ***The term chromosome refers to a numerical value or values that represent a candidate solution to the problem that the genetic algorithm is trying to solve.***

- Each candidate solution is encoded as an array of parameter values, a process that is also found in other optimization algorithms.
- If a problem has Npar dimensions, then typically each chromosome is encoded as an Npar-element array
- $$chromosome = [p1, p2, \ldots, pNpar\ ] \qquad (6.1.1)$$

- where each pi is a particular value of the ith parameter.

6.1.4. Operators of Genetic Algorithms

- *It is up to the creator of the GA to devise how to translate the sample space of candidate solutions into chromosomes.*
- *One approach is to convert each parameter value into a bit string (sequence of 1's and 0's), then concatenate the parameters end-to-end like genes in a DNA strand to create the chromosomes.*
- *Historically, chromosomes were typically encoded this way, and it remains a suitable method for discrete solution spaces.*
- *Modern computers allow chromosomes to include permutations, real numbers, and many other objects.*

6.1.4. Operators of Genetic Algorithms

- *A GA begins with a randomly chosen assortment of chromosomes, which serves as the first generation (initial population).*
- *Then each chromosome in the population is evaluated by the fitness function to test how well it solves the problem at hand.*
- *Now the selection operator chooses some of the chromosomes for reproduction based on a probability distribution defined by the user.*
- *The fitter a chromosome is, the more likely it is to be selected.*

- For example, if f is a non-negative fitness function, then the probability that chromosome C53 is chosen to reproduce might be

6.1.4. Operators of Genetic Algorithms

- $$P(C_{53}) = \left| \frac{f(C_{53})}{\sum_{i=1}^{N_{pop}} f(C_i)} \right| . \qquad (6.1.2)$$

- *Note that the selection operator chooses chromosomes with replacement, so the same chromosome can be chosen more than once.*
- The crossover operator mimics the biological crossing over and recombination of chromosomes.
- *This operator swaps a subsequence of two of the chosen chromosomes to create two offspring.*
- For example, if the parent chromosomes

6.1.4. Operators of Genetic Algorithms

- [*1101***0111001000**] and [0101***1101010010***] (6.1.3)
- are crossed over after the fourth bit, then
- [0101**0111001000**] and [*1101****1101010010***] (6.1.4)
- will be their offspring.

6.1.4. Operators of Genetic Algorithms

- ***The mutation operator randomly flips individual bits in the new chromosomes (turning a 0 into a 1 and vice versa).***
- *Typically, mutation happens with a very low probability, such as 0.001.*
- ***Some algorithms implement the mutation operator before the selection and crossover operators; this is a matter of preference.***
- The mutation operator plays an important role, even if it is secondary to those of selection and crossover.
- Selection and crossover maintain the genetic information of fitter chromosomes, but these chromosomes are only fitter relative to the current generation.

6.1.4. Operators of Genetic Algorithms

- ***This can cause the algorithm to converge too quickly and lose" potentially useful genetic material (1's or 0's at particular locations)".***
- *In other words, the algorithm can get stuck at a local optimum before finding the global optimum.*
- ***The mutation operator helps protect against this problem by maintaining diversity in the population, but it can also make the algorithm converge more slowly.***
- *Typically, the selection, crossover, and mutation process continues until the number of offspring is the same as the initial population, so that the second generation is composed entirely of new offspring and the first generation is completely replaced.*

6.1.4. Operators of Genetic Algorithms

- *Now the second generation is tested by the fitness function, and the cycle repeats.*

- *It is a common practice to record the chromosome with the highest fitness (along with its fitness value) from each generation, or the “best-so-far" chromosome.*
- *GAs are iterated until the fitness value of the “best-so-far" chromosome stabilizes and does not change for many generations.*
- *This means the algorithm has converged to a solution(s).*
- ***The whole process of iterations is called a run.***
- At the end of each run there is usually at least one chromosome that is a highly fit solution to the original problem.
- *Depending on how the algorithm is written, this could be the most fit of all the “best-so-far“ chromosomes, or the most fit of the final generation.*

6.1.4. Operators of Genetic Algorithms

- *The “performance" of a GA depends highly on the method used to encode candidate solutions into chromosomes and “the particular criterion for success," or what the fitness function is actually measuring.*

- *Other important details are the probability of crossover, the probability of mutation, the size of the population, and the number of iterations.*

- *These values can be adjusted after assessing the algorithm's performance on a few trial runs.*

6.1.4. Operators of Genetic Algorithms

- A generalized pseudo-code for a GA can be given by −
- *GA()*
- *initialize population*
- *find fitness of population*
- *while (termination criteria is reached) do*
- *parent selection*
- *crossover with probability pc*
- *mutation with probability pm*
- *decode and fitness calculation*
- *survivor selection*
- *find best*
- *return best*

6.1.5. Complexity of Genetic Algorithms

- **Time complexity of Genetic Algorithms**
- The time complexity of genetic algorithms is not a single, fixed value, but rather depends on several factors and the specific implementation.
- It is generally expressed in terms of Big-O notation, which describes the upper bound of the algorithm's growth rate in the worst-case scenario.
- ***Key factors influencing the time complexity include:***
- *Number of Generations (G): The total number of iterations the algorithm runs.*
- *Population Size (N): The number of individuals in each generation.*
- *Length of Genotypes (L): The number of genes or features in each individual's representation.*

6.1.5. Complexity of Genetic Algorithms

- *Complexity of the Fitness Function (f): The most significant factor, as evaluating the fitness of each individual in the population for every generation can be computationally expensive. If the fitness function itself has a complexity of O(C), then the total complexity will be impacted by this.*
- ***Complexity of Genetic Operators:***
- *Selection: Methods like tournament selection are often O(N), while roulette wheel selection might involve sorting, leading to O(N log N).*
- *Crossover and Mutation: These operations typically involve manipulating the genotypes and are often O(N * L) for the entire population.*

6.1.5. Complexity of Genetic Algorithms

- **General Approximation**
- Assuming a simple fitness function with O(1) complexity and common genetic operators, the time complexity of a generational genetic algorithm can be approximated as:

- $O(G * (N * C + N * \log N + N * L))$, (6.1.5)

- where:
- G is the number of generations.
- N is the population size.
- C is the complexity of the fitness function (often the dominant factor).
- log N arises from sorting in some selection methods.
- L is the length of the genotypes for crossover and mutation.

6.1.5. Complexity of Genetic Algorithms

- In many practical scenarios, especially when the fitness function is complex, the term N * C dominates the calculation, and the overall complexity is heavily influenced by the cost of evaluating the fitness function across the entire population for each generation.
- Therefore, the time complexity is often simplified to O(G * N * C), where C represents the complexity of a single fitness evaluation.

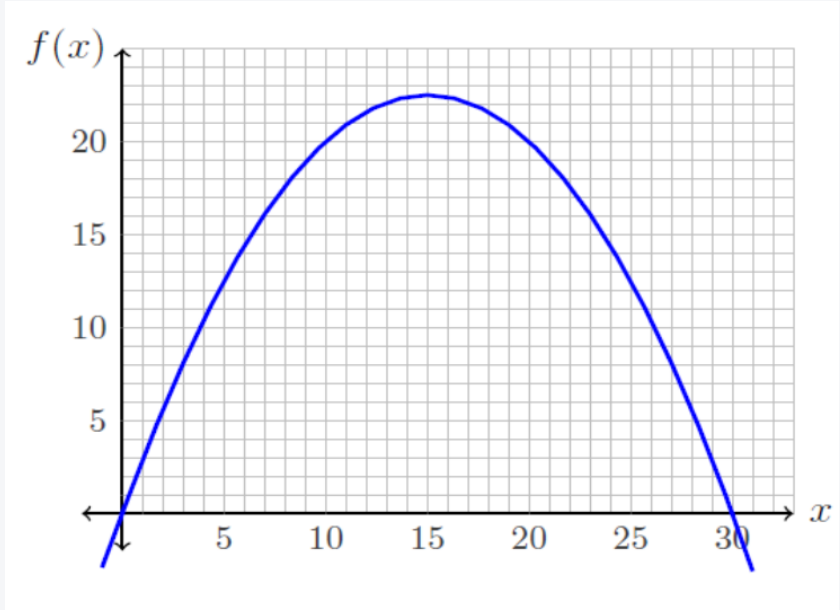
- **Space complexity of Genetic Algorithms**
- The space complexity for a GA is generally at least O(N * C), as it needs to store the population of N individuals, each of size C.
- This can increase if additional data structures are used.

6.1.6. Examples

- **Example 1:**
- **Maximizing a Function of One Variable**
- This example adapts the method of an example presented in [2].
- Consider the problem of maximizing the function
- $$f(x) = \frac{-x^2}{10} + 3x \qquad (6.1.6)$$
- where 0<=x<=31. This function is displayed in Fig. 6.1.10.

- [2] Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning. Reading: Addison-Wesley.

6.1.6. Examples



- Fig. 6.1.10.

6.1.6. Examples

- To solve this using a GA, we must encode the possible values of x as chromosomes.
- For this example, we will encode x as a binary integer of length 5.
- Thus, the chromosomes for our GA will be sequences of 0's and 1's with a length of 5 bits and have a range from 0 (00000) to 31 (11111).

6.1.6. Examples

- To begin the algorithm, we select an initial population of 10 chromosomes at random.
- We can achieve this by tossing a fair coin 5 times for each chromosome, letting heads signify 1 and tails signify 0.

- The resulting initial population of chromosomes is shown in Table 6.1.1.
- Next, we take the x-value that each chromosome represents and test its fitness with the fitness function.
- The resulting fitness values are recorded in the third column of Table 6.1.1.

6.1.6. Examples

- Table 6.1.1.

Chromosome Number	Initial Population	x Value	Fitness Value $f(x)$	Selection Probability
1	01011	11	20.9	0.1416
2	11010	26	10.4	0.0705
3	00010	2	5.6	0.0379
4	01110	14	22.4	0.1518
5	01100	12	21.6	0.1463
6	11110	30	0	0
7	10110	22	17.6	0.1192
8	01001	9	18.9	0.1280
9	00011	3	8.1	0.0549
10	10001	17	22.1	0.1497
		Sum	147.6	
		Average	14.76	
		Max	22.4	

$$01011 = 0\times 2^4 + 1\times 2^3 + 0\times 2^2 + 1\times 2^1 + 1\times 2^0 = 11$$

6.1.6. Examples

- We select the chromosomes that will reproduce based on their fitness values, using the following selection probability:

$$P(\text{chromosome } i \text{ reproduces}) = \frac{f(x_i)}{\sum_{k=1}^{10} f(x_k)} \qquad (6.1.7)$$

- Since our population has 10 chromosomes and each `mating' produces 2 offspring, we need 5 “mattings to produce a new generation of 10 chromosomes.
- The selected chromosomes are displayed in Table 6.1.2.
- To create their offspring, a crossover point is chosen at random, which is shown in the table as a vertical line. Note that it is possible that crossover does not occur, in which case the offspring are exact copies of their parents.

6.1.6. Examples

- Table 6.1.2.

Chromosome Number	Mating Pairs	New Population	x Value	Fitness Value $f(x)$
5	01|100	01010	10	20
2	11|010	11100	28	5.6
4	0111|0	01111	15	22.5
8	0100|1	01000	8	17.6
9	0001|1	01010	10	20
2	1101|0	11011	27	8.1
7	10110	10110	22	17.6
4	01110	01110	14	22.4
10	100|01	10001	17	22.1
8	010|01	01001	9	18.9
			Sum	174.8
			Average	17.48
			Max	22.5

6.1.6. Examples

- Lastly, each bit of the new chromosomes mutates with a low probability.
- For this example, we let the probability of mutation be 0.001.
- With 50 total transferred bit positions, we expect 50x0.001 = 0.05 bits to mutate.
- Thus, it is likely that no bits mutate in the second generation.
- For the sake of illustration, we have mutated one bit in the new population, which is shown in bold in Table 6.1.2.

6.1.6. Examples

- *After selection, crossover, and mutation are complete, the new population is tested with the fitness function.*
- *The fitness values of this second generation are listed in Table 6.1.2.*
- Although drawing conclusions from a single trial of an algorithm based on probability is, ”at best, a risky business," this example illustrates how genetic algorithms `evolve' toward fitter candidate solutions.
- *Comparing Table 6.1.2 to Table 6.1.1, we see that both the maximum fitness and average fitness of the population have increased after only one generation.*

6.1.6. Examples

- **Example 2:**
- Given a target string, the goal is to produce target string starting from a random string of the same length.

- In the following implementation, following analogies are made –

- Characters A-Z, a-z, 0-9, and other special symbols are considered as genes

6.1.6. Examples

- The python code of GAs and output results can be presented as follows:

```
# Python3 program to create target string, starting from
# random string using Genetic Algorithm

import random

# Number of individuals in each generation
POPULATION_SIZE = 100

# Valid genes
GENES = '''abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOP
QRSTUVWXYZ 1234567890, .-;:_!"#%&/()=?@${[]}'''

 @classmethod
    def mutated_genes(self):
        '''
        create random genes for mutation
        '''
        global GENES
        gene = random.choice(GENES)
        return gene
```

6.1.6. Examples

```
@classmethod
    def create_gnome(self):
        '''
        create chromosome or string of genes
        '''
        global TARGET
        gnome_len = len(TARGET)
        return [self.mutated_genes() for _ in range(gnome_len)]

def mate(self, par2):
        '''
        Perform mating and produce new offspring
        '''

        # chromosome for offspring
        child_chromosome = []
        for gp1, gp2 in zip(self.chromosome, par2.chromosome):
```

6.1.6. Examples

```
# random probability
            prob = random.random()

            # if prob is less than 0.45, insert gene
            # from parent 1
            if prob < 0.45:
                child_chromosome.append(gp1)

            # if prob is between 0.45 and 0.90, insert
            # gene from parent 2
            elif prob < 0.90:
                child_chromosome.append(gp2)

                 # otherwise insert random gene(mutate),
            # for maintaining diversity
            else:
                child_chromosome.append(self.mutated_genes())

        # create new Individual(offspring) using
        # generated chromosome for offspring
        return Individual(child_chromosome)
```

6.1.6. Examples

```
    def cal_fitness(self):
        '''
        Calculate fitness score, it is the number of
        characters in string which differ from target
        string.
        '''
        global TARGET
        fitness = 0
        for gs, gt in zip(self.chromosome, TARGET):
            if gs != gt: fitness+= 1
        return fitness

    # Driver code
def main():
    global POPULATION_SIZE

    #current generation
    generation = 1

    found = False
    population = []
```

6.1.6. Examples

```
# create initial population
   for _ in range(POPULATION_SIZE):
               gnome = Individual.create_gnome()
               population.append(Individual(gnome))

   while not found:

       # sort the population in increasing order of fitness score
       population = sorted(population, key = lambda x:x.fitness)

# if the individual having lowest fitness score i.e.
       # 0 then we know that we have reached to the target
       # and break the loop
       if population[0].fitness <= 0:
           found = True
           break

       # Otherwise generate new offsprings for new generation
       new_generation = []
```

6.1.6. Examples

```
# Perform Elitism, that mean 10% of fittest population
        # goes to the next generation
        s = int((10*POPULATION_SIZE)/100)
        new_generation.extend(population[:s])

# From 50% of fittest population, Individuals
        # will mate to produce offspring
        s = int((90*POPULATION_SIZE)/100)
        for _ in range(s):
            parent1 = random.choice(population[:50])
            parent2 = random.choice(population[:50])
            child = parent1.mate(parent2)
            new_generation.append(child)

            population = new_generation

        print("Generation: {}\tString: {}\tFitness: {}".\
              format(generation,
              "".join(population[0].chromosome),
              population[0].fitness))
```

6.1.6. Examples

```
        generation += 1

 print("Generation: {}\tString: {}\tFitness: {}".\
          format(generation,
          "".join(population[0].chromosome),
          population[0].fitness))

if __name__ == '__main__':
    main()
```

6.1.6. Examples

```
    Output

Generation: 1   String: 4YyQGNMW;ekc[!R9u Le    Fitness: 18
Generation: 2   String: 4YyQGNMW;ekc[!R9u Le    Fitness: 18
Generation: 3   String: I(ljdDig ,eI#3r MN8U                    Fitness: 17
Generation: 4   String: I(ljdDig ,eI#3r MN8U                    Fitness: 17
Generation: 5   String: 7(To6  = ek() YBD 3Y                    Fitness: 16
Generation: 6   String: I(lodjig;MkTW3r  D ek                   Fitness: 15
Generation: 7   String: 7 R50D seeki#)7cDVkV    Fitness: 14
Generation: 8   String: #}lod0 6KekW))r@D/kb    Fitness: 13
Generation: 9   String: #}lod0 6KekW))r@D/kb    Fitness: 13
Generation: 10  String: I loGs x;ek9?#r@D kk    Fitness: 11
Generation: 11  String: I loGs x;ek9?#r@D kk    Fitness: 11
Generation: 12  String: I loGs x;ek9?#r@D kk    Fitness: 11
Generation: 13  String: I loXs 0eek9#.rGDVsV    Fitness: 10
Generation: 14  String: I loXs 0eek9#.rGDVsV    Fitness: 10
Generation: 15  String: I loXs 0eek9#.rGDVsV    Fitness: 10
Generation: 16  String: I lovC geeka#jrver,V    Fitness: 9
Generation: 17  String: I )ooN geekk?jrGeekV    Fitness: 8
Generation: 18  String: I )ooN geekk?jrGeekV    Fitness: 8
Generation: 19  String: I )ooN geekk?jrGeekV    Fitness: 8
Generation: 20  String: I )ooN geekk?jrGeekV    Fitness: 8
Generation: 21  String: I lovC Ge6kK#)rGeAkk    Fitness: 7
Generation: 22  String: I lovC Ge6kK#)rGeAkk    Fitness: 7
Generation: 23  String: I lovC Ge6kK#)rGeAkk    Fitness: 7
Generation: 24  String: I lovC Ge6kK#)rGeAkk    Fitness: 7
Generation: 25  String: I loRC Ieekpf)rGeek3    Fitness: 6
Generation: 26  String: I lovC Geek-fbrGe}kV    Fitness: 5
Generation: 27  String: I lovC Geek-fbrGe}kV    Fitness: 5
Generation: 28  String: I lovC Geek-fbrGe}kV    Fitness: 5
Generation: 29  String: I lovC GeekQfo Geekk    Fitness: 4
Generation: 30  String: I lovC GeekQfo Geekk    Fitness: 4
```

6.1.6. Examples

```
Generation: 31  String: I lovC GeekQfo Geekk    Fitness: 4
Generation: 32  String: I lovC GeekQfo Geekk    Fitness: 4
Generation: 33  String: I lovC GeekQfo Geekk    Fitness: 4
Generation: 34  String: I lovk Geek-forGeekA    Fitness: 3
Generation: 35  String: I lovk Geek-forGeekA    Fitness: 3
Generation: 36  String: I lovk Geek-forGeekA    Fitness: 3
Generation: 37  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 38  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 39  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 40  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 41  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 42  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 43  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 44  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 45  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 46  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 47  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 48  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 49  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 50  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 51  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 52  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 53  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 54  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 55  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 56  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 57  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 58  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 59  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 60  String: I lov  GeeksforGeekt    Fitness: 2
```

6.1.6. Examples

```
Generation: 61  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 62  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 63  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 64  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 65  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 66  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 67  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 68  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 69  String: I lov  GeeksforGeekt    Fitness: 2
Generation: 70  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 71  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 72  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 73  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 74  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 75  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 76  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 77  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 78  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 79  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 80  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 81  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 82  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 83  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 84  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 85  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 86  String: I lov. GeeksforGeeks    Fitness: 1
Generation: 87  String: I love GeeksforGeeks    Fitness: 0
```

6.1.6. Examples

- **Note:** Every-time algorithm starts with random strings, so output may differ.

- As we can see from the output, our algorithm sometimes stuck at a local optimum solution, this can be further improved by updating fitness score calculation algorithm or by mutation and crossover operators.

6.1.7. Advantages and Limitations of Genetic Algorithms

- **GAs advantages**
- *Does not require any derivative information.*
- *Is faster and more efficient as compared to the traditional methods.*
- *Has good parallel capabilities.*
- *Optimizes both continuous and discrete functions and multi-objective problems.*
- *Provides a list of "good" solutions and not just a single solution.*
- *Always gets an answer to the problem, which gets better over the time.*
- *Useful when the search space is very large and there are large number of parameters involved.*

6.1.7. Advantages and Limitations of Genetic Algorithms

- **GAs limitations**
- *GAs are not suited for all problems, especially problems which are simple and for which derivative information is available.*
- *Fitness value is calculated repeatedly which might be computationally expensive for some problems.*
- *Being stochastic, there are no guarantees on the optimality or the quality of the solution.*
- *If not implemented properly, the GA may not converge to the optimal solution.*

6.1.8. Applications of Genetic Algorithms

- **GA applications**
- ***Optimization*** *– GAs are most commonly used in optimization problems wherein we have to maximize or minimize a given objective function value under a given set of constraints.*
- ***Economics*** *– GAs are also used to characterize various economic models, game theory, asset pricing, etc.*
- ***Neural Networks*** *– GAs are also used to train neural networks, particularly recurrent neural networks.*
- ***Parallelization*** *– GAs also have very good parallel capabilities and prove to be very effective means in solving certain problems and provide a good area for research.*
- ***Image Processing*** *– GAs are used for various digital image processing (DIP) tasks as well like dense pixel matching.*

6.1.8. Applications of Genetic Algorithms

- ***Vehicle routing problems*** *− With multiple soft time windows, multiple depots and a heterogeneous fleet.*
- ***Scheduling applications*** *− GAs are used to solve various scheduling problems as well, particularly the time tabling problem.*
- ***Machine Learning*** *− as already discussed, genetic based machine learning (GBML) is a niche area in machine learning.*
- ***Robot Trajectory Generation*** *− GAs have been used to plan the path which a robot arm takes by moving from one point to another.*
- ***Parametric Design of Aircraft*** *− GAs have been used to design aircrafts by varying the parameters and evolving better solutions.*
- ***DNA Analysis*** *− GAs have been used to determine the structure of DNA using spectrometric data about the sample.*

6.1.8. Applications of Genetic Algorithms

- ***Multimodal Optimization*** *– GAs are obviously very good approaches for multimodal optimization in which we have find multiple optimum solutions.*
- ***Traveling salesman problem and its applications*** *– GAs have been used to solve the TSP, which is a well-known combinatorial problem using novel crossover and packing strategies.*